

SHILATE: SOFTWARE-DEFINED VEHICLE REINFORCEMENT LEARNING — FROM SIMULATION TO SDV DEPLOYMENT

Candidate: Given Name FAMILY NAME

Scientific coordinator: Professor Given Name FAMILY NAME

Session: June 2026

ABSTRACT

The proliferation of software-defined vehicle (SDV) platforms has opened an opportunity to combine modern deep reinforcement learning (RL) with real automotive runtime stacks. This thesis presents **SHILATE** — a complete, open-source pipeline that trains a Proximal Policy Optimization (PPO) agent inside a Unity physics simulation and deploys the resulting policy onto an Eclipse Leda SDV stack, bridging the gap between laboratory RL research and realistic vehicle software deployment.

The simulation component is built in Unity 2022 LTS. A single rear-wheel-drive vehicle navigates a circular obstacle course instrumented with a 21-ray LiDAR-style sensor. The training bridge computes a shaped reward signal from progress, collision events, and episode milestones, and communicates with the Python training process exclusively over MQTT, keeping the two subsystems fully decoupled. A dedicated Editor window (*SHILATE Training Controller*) provides real-time health monitoring, metric graphs, and one-click training control without leaving the Unity Editor.

The Python training stack wraps the simulation in a Gymnasium-compliant environment, configures Stable-Baselines3 PPO with a two-layer actor-critic network, and streams structured telemetry back to the Editor. Trained checkpoints are subsequently deployed to the Eclipse Leda SDV image via a containerised MQTT-to-Kuksa bridge and a Velocitas vehicle application that enforces operational limits (overspeed, RPM redline) on the running agent.

Experimental results demonstrate that the PPO agent converges reliably on the obstacle course within 100 000 interaction steps and that the end-to-end deployment chain correctly propagates vehicle telemetry from the simulation through the VSS data layer to the Velocitas safety monitor. The project shows that an integrated sim-to-SDV pipeline is achievable with commodity open-source tooling, and identifies curriculum learning and multi-environment parallelisation as natural extensions for future work.

Keywords: reinforcement learning, software-defined vehicle, Unity simulation, PPO, MQTT, Eclipse Leda, Kuksa, Stable-Baselines3, Gymnasium.

CONTENTS

1	Introduction	4
1.1	$\LaTeX$ Instructions	4
1.1.1	Structure	4
1.1.2	Authenticity Declaration	4
1.1.3	Other $\LaTeX$ elements	4
1.2	General Information	5
2	Bibliography	6

LIST OF FIGURES

LIST OF TABLES

1. INTRODUCTION

1.1 MOTIVATION AND CONTEXT

The automotive industry is undergoing a fundamental architectural shift. Traditional vehicles are increasingly giving way to software-defined vehicles (SDVs) — platforms in which electronic control units are consolidated onto high-performance compute hardware and the vehicle's behaviour is governed by software layers that can be updated over the air **EclipseLeda2023**. This transformation enables capabilities that were previously inconceivable: dynamic reconfiguration of driving characteristics, continuous safety-critical updates without a workshop visit, and the integration of machine-learning-based components directly into the vehicle runtime.

At the same time, deep reinforcement learning (RL) has matured into a practical engineering tool. Algorithms such as Proximal Policy Optimization (PPO) **Schulman2017** achieve human-competitive or superhuman performance in continuous control tasks, with comparatively modest tuning effort. The natural question that arises is: *how can an RL-trained policy be moved, end-to-end, from a training simulation onto an SDV runtime stack in a reproducible and maintainable way?*

This question is not merely academic. Automotive-grade deployment requires that a trained model run inside a standardised software container, interact with vehicle signals through a well-defined data layer, and remain subject to functional-safety monitors that can intervene when the model operates outside its intended envelope. Bridging this gap between laboratory RL and production SDV middleware is the core motivation behind the work presented in this thesis.

1.2 PROBLEM STATEMENT

Existing RL research in the autonomous driving domain concentrates almost exclusively on training performance — reward curves, sample efficiency, and policy robustness **Kiran2021**. The software engineering challenge of taking a trained checkpoint and deploying it as a first-class citizen of an SDV runtime stack is largely unexplored in the open-source community. Concretely, the following sub-problems remain unaddressed in an integrated, open-source pipeline:

1. **Simulation-to-middleware coupling.** A physics simulator and an SDV middleware stack speak fundamentally different protocols; a communication bridge that is both low-latency and decoupled from either system is needed.
2. **Standardised vehicle-signal representation.** Raw simulator outputs must be mapped onto industry-standard Vehicle Signal Specification (VSS) paths so that the SDV safety layer can interpret them without simulator-specific knowledge.
3. **Operational safety monitoring.** A deployed RL policy may extrapolate outside its training distribution; a lightweight, declarative safety monitor must be able to flag

or override the policy's actions in real time.

4. **Reproducible training workflow.** The entire pipeline — from launching the simulation to saving a deployable checkpoint — must be reproducible from a single control surface without manual orchestration.

1.3 OBJECTIVES

This thesis addresses the above problems through the design, implementation, and evaluation of **SHILATE** (Software-defined-vehicle Hardware-In-the-Loop Autonomous Training Environment). The specific objectives are:

1. Design and implement a Unity-based physics simulation of a rear-wheel-drive vehicle navigating an instrumented obstacle course.
2. Define a shaped reward function that guides a PPO agent to complete laps reliably and safely, and expose a Gymnasium-compliant Python interface over MQTT.
3. Implement a real-time health-monitoring and training-control layer inside the Unity Editor so that a researcher can observe and intervene in a training run without leaving the development environment.
4. Construct an SDV deployment chain using Eclipse Leda, the Kuksa vehicle signal databroker, and a Velocitas vehicle application that enforces operational limits on the running policy.
5. Evaluate the pipeline quantitatively: training convergence, inference lap performance, and end-to-end signal propagation through the SDV stack.

1.4 THESIS STRUCTURE

The remainder of this document is organised as follows. ?? reviews theoretical background in reinforcement learning, SDV middleware, and communication protocols, and situates SHILATE among related work. ?? presents the overall system architecture. ?? describes the Unity simulation in detail. ?? documents the Python RL training pipeline. ?? explains the SDV integration through Eclipse Leda. ?? reports experimental results. ?? concludes with a summary of contributions and future work.

1.5 L^AT_EX INSTRUCTIONS

1.5.1 STRUCTURE

Each file belonging to the document can be found in `bachelors_en/`. The main file of the template is `main.tex`, which must be compiled using `pdflatex` (usually the default option). Files in `components/` define the template and **should not be modified**.

In `customs.tex` reside the personal data which appear in places not directly accessible by the user and should be modified. Every file in `chapters/` represents a chapter where the content of the thesis is written. Under `images/` will be placed all the images used in the document.

1.5.2 AUTHENTICITY DECLARATION

The blank declaration can be obtained in two ways:

- from `main.tex` residing in `authenticity_declaration/`, an unsigned variant is created after filling personal data in `customs.tex`, similarly with the structure of Section 1.1.1.
- from the document at <https://ac.upt.ro/finalizare-studii/>, in section “Documente importante”. This should be manually filled with personal data.

Regardless of the option chosen, the document must be printed, **physically signed(with a pen)** and scanned in order to be attached. In `customs.tex`, the path must be changed from the example to the signed and scanned PDF document, which will be saved in `bachelors_ro/`.

1.5.3 OTHER L^AT_EX ELEMENTS

References to other elements in the document (excepting citations and links): Chapter 1; ??.

For each reference, a `\label` is needed, which will be inserted anywhere in the text or immediately after chapter/section titles, and in figures/tables/code fragments.

Citations will be added using `bibtex`, example: ??. Inserting images (??), tables (??) and code fragments (??) will be added using the corresponding code. For tables, an external formatting tool is recommended: (example ??).

1.6 GENERAL INFORMATION

Each chapter must have a clear structure, will begin on a new page and will have a title. It will be followed by two blank 12 pt. lines.

Each sub-section title (e.g. 1.2 General information) shall begin after a 12 points blank line after the text and shall be followed by a 12 points blank line.

The text of the paper will be justified. It is recommended to check the spelling of the text with the help of the speller of the Word program. It is recommended that the thesis paper should not exceed a number of 100 pages, annexes included.

Rules applying to the text of the paper:

- a. Margins of the page – the following values will be used:
 - interior: 2 cm
 - exterior: 2 cm
 - up: 2,5 cm (header included)
 - down: 2 cm
- b. Line spacing – the text shall apply a 1.15 line spacing

- c. Indentation – text within regular paragraphs will be aligned between the left and the right margins (justified). The first line of each paragraph will have a 1.5 cm indentation. There will be an exception for chapter titles, which will be aligned left, just like the titles of the tables and of the figures (see explanations below);
- d. Font – the 12 points Nimbus Sans font will be used, as well as the specific diacritics of the paper language (ex: ă, ș, ț, î, â - for the Romanian language);
- e. Page numbering - Page numbering runs from the title page to the last page of the paper, but the page number appears starts on the Introduction page. The page number is inserted at the bottom of the page, centered.
- f. f. Page header – it shall be inserted starting with the introduction page and contains, in successive lines, the following text, aligned left and with the size of 8 points: (i) the text Politehnica University of Timișoara ; (ii) the name of the program of study and the year of the paper defense; (iii) the name of the candidate (left) and the title of the paper. At the right of the header, the UPT logo may be inserted;

2. BACKGROUND AND RELATED WORK

2.1 REINFORCEMENT LEARNING FUNDAMENTALS

Reinforcement learning is a computational framework for sequential decision making under uncertainty **Sutton2018**. An agent interacts with an environment modelled as a Markov Decision Process (MDP), defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ is the transition kernel, $R : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the reward function, and $\gamma \in [0, 1)$ is the discount factor. The agent's goal is to learn a policy $\pi : \mathcal{S} \rightarrow \mathcal{A}$ that maximises the expected discounted return $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$.

Policy gradient methods optimise the policy directly via the gradient of the expected return with respect to policy parameters θ . The fundamental result is the policy gradient theorem:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a|s) \cdot Q^{\pi_{\theta}}(s, a)]. \quad (2.1)$$

Actor-critic architectures maintain both a policy network (actor) and a value network (critic) trained simultaneously; a learned baseline $V_{\phi}(s)$ reduces gradient variance through the advantage estimate $A(s, a) = Q(s, a) - V_{\phi}(s)$.

2.2 PROXIMAL POLICY OPTIMIZATION

Proximal Policy Optimization **Schulman2017** takes the largest possible improvement step on a policy without destabilising training. It replaces Trust Region Policy Optimization's constrained optimisation with a clipped surrogate objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon) \hat{A}_t \right) \right], \quad (2.2)$$

where $r_t(\theta) = \pi_{\theta}(a_t|s_t) / \pi_{\theta_{\text{old}}}(a_t|s_t)$ is the probability ratio, $\hat{A}_t$ is the estimated advantage, and ε (typically 0.2) is the clipping threshold. The full objective adds a value-function regression term and an entropy bonus:

$$L(\theta) = L^{\text{CLIP}}(\theta) - c_1 L^{\text{VF}}(\theta) + c_2 H[\pi_{\theta}(\cdot|s_t)]. \quad (2.3)$$

PPO is well suited to the vehicle control domain because its continuous Gaussian policy maps naturally onto a steer/throttle/brake action space, and its on-policy rollout structure integrates cleanly with a wall-clock-rate simulation running in a separate process. The Stable-Baselines3 library **Raffin2021** provides the production implementation used in this project, validated against reference benchmarks.

2.3 SIMULATION-TO-REAL TRANSFER IN AUTONOMOUS DRIVING

Training autonomous driving policies in simulation rather than on physical hardware is standard practice, motivated by safety, cost, and the ability to reset episodes deterministically **Kiran2021**. The primary risk is the *sim-to-real gap*: discrepancies between simulated and

real sensor readings, physics, and environmental conditions cause policies that perform well in simulation to degrade on deployment.

Prominent platforms include CARLA **Dosovitskiy2017**, an Unreal-Engine-based open-source simulator modelling urban driving at high visual fidelity, and the Unity ML-Agents Toolkit **Juliani2020**, which provides a general-purpose RL training framework tightly integrated with the Unity Editor. SHILATE uses raw Unity physics without ML-Agents for two reasons: it avoids an opaque intermediate API layer, and the training process communicates exclusively through MQTT — the same protocol used by the target SDV stack — so no additional bridge is required for deployment.

2.4 SOFTWARE-DEFINED VEHICLES AND THE VSS DATA MODEL

The COVESA Vehicle Signal Specification (VSS) **COVESA2023** is a hierarchical, vendor-neutral data model for vehicle signals. VSS paths follow a dotted-name convention, e.g. `Vehicle.Speed` (km/h), `Vehicle.Powertrain.CombustionEngine.Speed` (RPM), and `Vehicle.Chassis.SteeringWheel.Angle` (degrees).

The Eclipse Kuksa databroker **EclipseKuksa2024** implements a VSS-aligned gRPC server acting as the vehicle signal hub. Producers write to VSS paths; consumers subscribe to changes. Eclipse Velocitas **EclipseVelocitas2024** builds on Kuksa with a vehicle-application SDK and deployment scaffold. In SHILATE, the Velocitas application monitors the running RL agent for speed and RPM limit violations.

2.5 MQTT AND PUBLISH-SUBSCRIBE MESSAGING

MQTT 3.1.1 **OASIS2014** is a lightweight publish-subscribe protocol designed for constrained IoT environments. A central broker decouples producers and consumers: a publisher pushes a message to a topic string and all subscribers to that topic receive it with no direct network coupling between the two parties. SHILATE uses QoS 0 (at-most-once delivery) throughout, which minimises latency at the cost of possible message loss — acceptable in a training loop where the next observation replaces the current one within tens of milliseconds. The MQTT protocol is also natively supported by the Eclipse Leda SDV image, which ships Mosquitto as a core service.

2.6 RELATED WORK

A substantial body of work addresses RL for autonomous driving within simulation **Kiran2021**. OpenAI's `gym-car-racing` **Brockman2016** provides a top-down racetrack benchmark but offers no path to real vehicle deployment. CARLA-based RL studies **Dosovitskiy2017** achieve high visual realism but operate as closed systems with no integration to SDV middleware. The Unity ML-Agents toolkit **Juliani2020** enables RL training within Unity but couples the Python trainer tightly to Unity's side-channel API, making it difficult to reuse the same environment against a physical vehicle.

From the SDV side, the Eclipse Kuksa and Velocitas communities have demonstrated containerised vehicle applications that read VSS signals and enforce policies, but without an

RL training pipeline attached. SHILATE contributes the missing link: a training-to-deployment chain that is agnostic to both the simulation engine and the SDV stack as long as both speak MQTT and VSS.

3. BIBLIOGRAPHY

- [1] A. Einstein, “Zur Elektrodynamik bewegter Körper. (German) [On the electrodynamics of moving bodies],” *Annalen der Physik*, vol. 322, no. 10, pp. 891–921, 1905. DOI: <http://dx.doi.org/10.1002/andp.19053221004>.

DECLARAȚIE DE AUTENTICITATE A LUCRĂRII DE FINALIZARE A STUDIILOR *

Subsemnatul **ION POPESCU**, legitimat cu **CI** seria **TZ** nr. **100772**, CNP **5000102355779**, autorul lucrării **TITLUL LUCRĂRII DE FINALIZARE A STUDIILOR** elaborată în vederea susținerii examenului de finalizare a studiilor de **LICENȚĂ** organizat de către Facultatea **AUTOMATICĂ ȘI CALCULATOARE** din cadrul Universității Politehnica Timișoara, sesiunea **IUNIE** a anului universitar **2022**, coordonator **dr.ing. MIHAI IONESCU**, luând în considerare conținutul art. 34 din *Regulamentul privind organizarea și desfășurarea examenelor de licență/diplomă și disertație*, aprobat prin HS nr. 109/14.05.2020 și cunoscând faptul că în cazul constatării ulterioare a unor declarații false, voi suporta sancțiunea administrativă prevăzută de art. 146 din Legea nr. 1/2011 – legea educației naționale și anume anularea diplomei de studii, declar pe proprie răspundere, că:

- această lucrare este rezultatul propriei activități intelectuale,
- lucrarea nu conține texte, date sau elemente de grafică din alte lucrări sau din alte surse fără ca acestea să nu fie citate, inclusiv situația în care sursa o reprezintă o altă lucrare/alte lucrări ale subsemnatului.
- sursele bibliografice au fost folosite cu respectarea legislației române și a convențiilor internaționale privind drepturile de autor.
- această lucrare nu a mai fost prezentată în fața unei alte comisii de examen de licență/diplomă/disertație.

Timișoara,

Data

Semnătura

*Declarația se completează „de mână” și se inserează în lucrarea de finalizare a studiilor, la sfârșitul acesteia, ca parte integrantă.